SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0607 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 40%

QUESTIONS: 03 **Total Marks:25**

Annexure A

Constraint Based Problem Solving

Code of Conduct:

I ***VIMALA K 192424276*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Vimala K

Signature of the student:

Criteria	*Conceptual Understanding*	*Accuracy of Answers*	*Application of Knowledge*	*Completeness of Response*	*Clarity & Presentation*	*Tota*
Weightage	*30%*	*25%*	*15%*	*15%*	*15%*	
Q1						
Q2						

Total Marks :

Annexure A

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Constraint-Based Data Cleaning Pipeline Using NumPy and Pandas

1. Objective

The objective is to clean a large patient health-record dataset stored in a CSV file by using NumPy and Pandas. The pipeline ensures that:

1. No missing values remain.
2. Numerical values are within medically valid ranges.
3. Categorical values are standardized.
4. Maximum original data integrity is preserved.

2. Data Cleaning Pipeline

```
import pandas as pd
import numpy as np

# Step 1: Load the dataset
df = pd.read_csv("patient_records.csv")

# Step 2: Standardize column names
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

# Step 3: Convert numerical columns
numeric_cols = ["age", "blood_pressure", "glucose", "heart_rate"]

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Step 4: Replace invalid values with NaN
df.loc[(df["age"] < 0) | (df["age"] > 120), "age"] = np.nan
df.loc[(df["blood_pressure"] < 40) | (df["blood_pressure"] > 250),
       "blood_pressure"] = np.nan
df.loc[(df["glucose"] < 20) | (df["glucose"] > 600), "glucose"] = np.nan
df.loc[(df["heart_rate"] < 30) | (df["heart_rate"] > 220),
```

```
    "heart_rate"] = np.nan

# Step 5: Fill missing numerical values using median
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

# Step 6: Standardize categorical fields
df["gender"] = (
    df["gender"]
    .astype(str)
    .str.strip()
    .str.lower()
    .replace({
        "m": "male",
        "f": "female",
        "man": "male",
        "woman": "female"
    })
)

df["smoking_status"] =
    ( df["smoking_status"]
    .astype(str)
    .str.strip()
    .str.lower()
    .replace({
        "yes": "smoker",
        "y": "smoker",
        "no": "non-smoker",
        "n": "non-smoker"
    })
)
```

```
# Step 7: Fill missing categorical values
categorical_cols = ["gender", "smoking_status"]

for col in categorical_cols:
    df[col] = df[col].replace("nan", np.nan)
    df[col] = df[col].fillna(df[col].mode()[0])

# Step 8: Final constraint check
print("Missing values:")
print(df.isnull().sum())

print("\nSummary statistics:")
print(df[numeric_cols].agg(["min", "max", "mean", "median"]))

# Step 9: Save cleaned dataset
df.to_csv("cleaned_patient_records.csv", index=False)
```

3. Use of Indexing

Indexing is used to locate and modify specific records that violate constraints.

```
df.loc[df["age"] > 120, "age"] = np.nan
```

Here, .loc[] identifies patients whose age is outside the valid range and marks the invalid value as missing before safe replacement.

4. Use of Filtering

Filtering identifies records satisfying particular medical constraints.

```
valid_patients = df[
    (df["age"] >= 0) &
    (df["age"] <= 120) &
    (df["glucose"] >= 20) &
    (df["glucose"] <= 600)
]
```

This ensures that only records satisfying the defined constraints are selected.

5. Use of Aggregation

Aggregation functions help identify abnormal values and calculate appropriate replacement values.

df[numeric_cols].agg(["min", "max", "mean", "median"])

The median is particularly useful for replacing missing numerical values because it is less affected by extreme outliers than the mean.

For categorical data:

df["gender"].value_counts()

This helps identify inconsistent categories and determine the most frequent category using mode().

5. Constraint Summary

Constraint	**Pandas/NumPy Technique**	**Action**
Missing values	isnull(), fillna()	Replace missing values
Invalid numerical values	.loc[], Boolean filtering	Convert invalid values to NaN
Outliers	Range filtering, median	Detect and replace extreme values
Categorical inconsistency	str.strip(), str.lower(), replace()	Standardize categories
Data validation	min(), max(), agg()	Verify numerical ranges
Category validation	value_counts(), unique()	Check standardized categories
Data preservation	Median/mode imputation	Avoid unnecessary row deletion

7. Conclusion

The proposed constraint-based cleaning pipeline uses NumPy and Pandas to systematically detect and correct missing, invalid, inconsistent, and extreme values. Indexing and filtering enforce medical constraints, while aggregation functions such as median(), mode(), min(), and max() support reliable validation and imputation. Instead of unnecessarily deleting patient records, invalid values are corrected or imputed, thereby preserving maximum data integrity while producing a complete and standardized dataset.

2. Memory-Efficient Numerical Data Processing Using NumPy

1. Objective

The objective is to process a massive numerical dataset on a system with limited memory while performing mathematical operations, random sampling, and statistical analysis. NumPy features such as efficient array creation, slicing, indexing, aggregation, and in-place operations can reduce memory consumption and improve performance.

2. Proposed Solution

```
import numpy as np
data =
    np.random.default_rng(42).random( (1000
    000, 10), dtype=np.float32
)
sample = data[:10000]
sample *= 2
data[data < 0.5] = 0
mean_value = np.mean(data, axis=0, dtype=np.float32)
sum_value = np.sum(data, axis=0, dtype=np.float32)
print("Mean:", mean_value)
print("Sum:", sum_value)
```

3. Efficient Array Creation

Use an appropriate data type according to the required precision.

data = np.zeros((1000000, 10), dtype=np.float32)

float32 requires 4 bytes per value, whereas float64 requires 8 bytes, so the memory requirement can be approximately halved when float32 is sufficient.

Functions such as zeros(), ones(), empty(), and arange() should be preferred over repeatedly growing arrays.

4. Slicing Instead of Copying

NumPy slicing generally creates a view of the original array rather than a complete copy.

subset = data[1000:5000]

This is more memory-efficient than:

subset = data[1000:5000].copy()

A copy should only be created when an independent array is actually required.

5. In-Place Operations

In-place operations reuse the existing memory instead of creating another large temporary array.

data += 10

data *= 2

data -= 5

Instead of:

data = data + 10

the in-place version can reduce temporary memory allocation.

6. Random Sampling

For large datasets, only the required number of samples should be selected.

```
rng = np.random.default_rng(42)

indices =
   rng.choice( data.shape[0],
   size=10000,
   replace=False
)

sample = data[indices]
```

If sampling is performed repeatedly, the sample size should be kept as small as the analysis requires.

7. Optimizing Indexing

Boolean indexing is useful for identifying values satisfying a condition:

indices = np.flatnonzero(data[:, 0] > 50)

For memory-sensitive operations, avoid creating several large temporary Boolean arrays simultaneously.

For example, process the data in chunks:

```
chunk_size = 100000
for start in range(0, len(data), chunk_size):
    end = min(start + chunk_size, len(data))
    chunk = data[start:end]
    chunk *= 2
    result = np.mean(chunk, axis=0)
    print(result)
```

This approach allows very large datasets to be processed using a smaller working-memory footprint.

8. Optimizing Aggregation

Aggregation can be performed along a specific axis instead of creating unnecessary intermediate arrays.

```
mean = np.mean(data, axis=0)
maximum = np.max(data, axis=0)
minimum = np.min(data, axis=0)
```

For very large datasets, aggregation can also be performed **chunk by chunk** and combined afterward.

9. Memory Optimization Techniques

Technique	Implementation	Benefit
Appropriate data type	dtype=np.float32	Reduces memory usage
Efficient creation	np.empty(), np.zeros()	Avoids repeated allocation
Slicing	data[1000:5000]	Creates a view
In-place operations	data *= 2	Avoids temporary arrays
Limited sampling	rng.choice()	Processes only required samples
Chunk processing	Process fixed-size blocks	Prevents memory overflow
Axis aggregation	np.mean(data, axis=0)	Efficient statistical analysis
Reuse arrays	Modify existing arrays	Reduces allocations

10. Debugging Memory Overflow

If a memory overflow occurs, first inspect the array size:

```
print(data.shape)
print(data.dtype)
print(data.nbytes / (1024**2), "MB")
```

nbytes shows how much memory the NumPy array occupies.

Other debugging strategies include:

- Reduce the array data type from float64 to float32 when acceptable.
- Process the dataset in chunks.
- Replace unnecessary copies with views.
- Use in-place operations.
- Avoid creating multiple large temporary arrays.
- Reduce the random sample size.
- Delete unused arrays using del.
- Use Python memory-profiling tools when the bottleneck is difficult to identify.

11. Debugging Performance Bottlenecks

Use timing to identify slow operations:

```
import time
start = time.perf_counter()
result = np.mean(data, axis=0)
end = time.perf_counter()
print("Execution time:", end - start)
```

If an operation is slow, check whether:

- A Python loop can be replaced with a NumPy vectorized operation.
- Unnecessary .copy() operations are being performed.
- The array has an inefficient data type.
- Excessive indexing or repeated allocation is occurring.
- The dataset can be processed in chunks.

12. Conclusion

A memory-efficient NumPy solution should use compact data types, array slicing, vectorized operations, in-place calculations, controlled random sampling, and chunk-based processing. Indexing should avoid unnecessary copies, while aggregation should operate directly along required axes. Monitoring nbytes, execution time, and unnecessary temporary arrays helps identify memory overflow and performance bottlenecks. Thus, even massive numerical datasets can be processed efficiently within limited memory constraints.